

Chapter 11 Inheritance and Polymorphism



Motivations

Suppose you will define classes to model circles, rectangles, and triangles. These classes have many common features. What is the best way to design these classes so to avoid redundancy? The answer is to use inheritance.

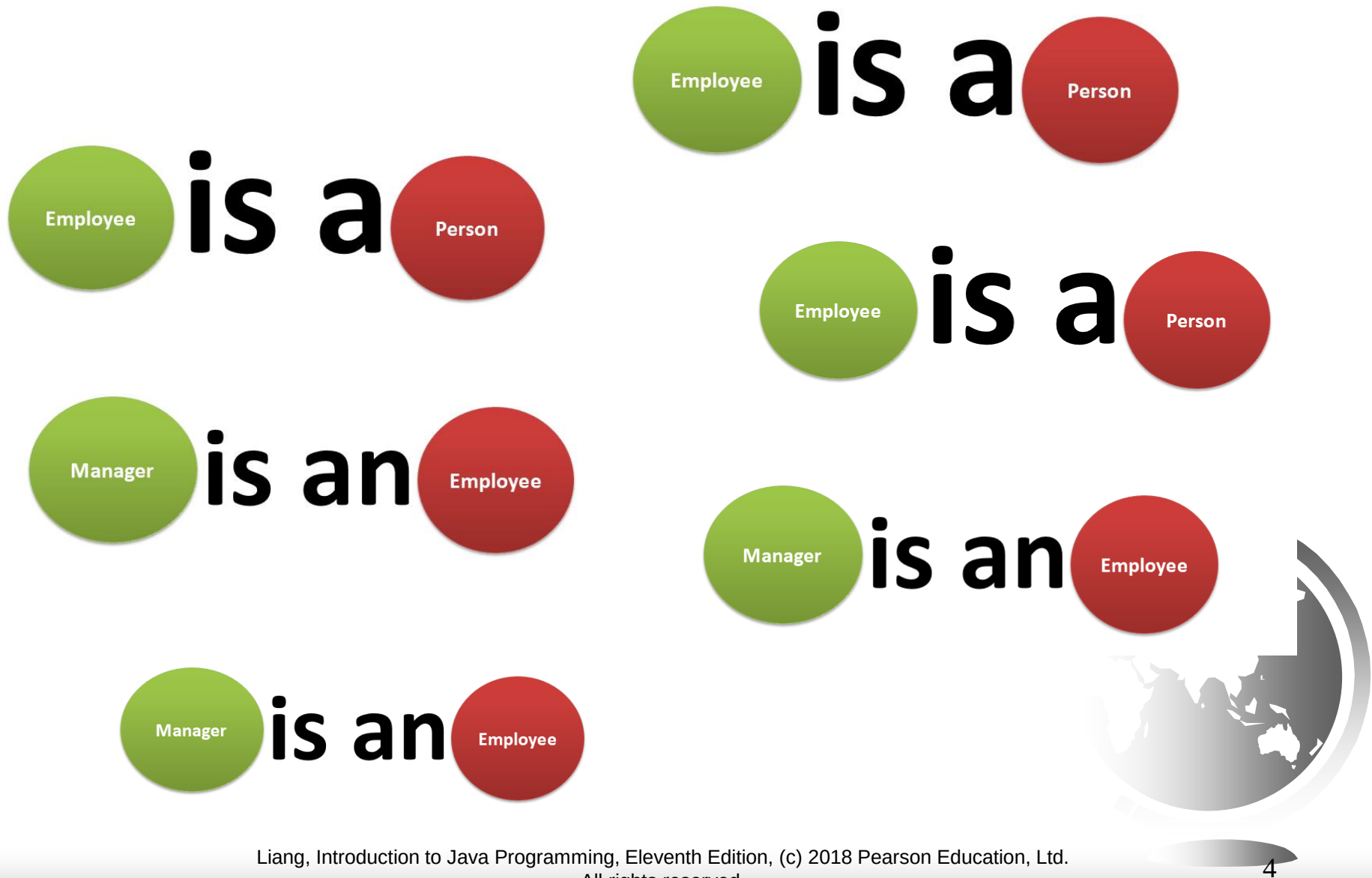


Objectives

- u To define a subclass from a superclass through inheritance (§11.2).
- u To invoke the superclass's constructors and methods using the **super** keyword (§11.3).
- u To override instance methods in the subclass (§11.4).
- u To distinguish differences between overriding and overloading (§11.5).
- u To explore the **toString()** method in the **Object** class (§11.6).
- u To discover polymorphism and dynamic binding (§§11.7–11.8).
- u To describe casting and explain why explicit downcasting is necessary (§11.9).
- u To explore the **equals** method in the **Object** class (§11.10).
- u To store, retrieve, and manipulate objects in an **ArrayList** (§11.11).
- u To implement a **Stack** class using **ArrayList** (§11.12).
- u To enable data and methods in a superclass accessible from subclasses using the **protected** visibility modifier (§11.13).
- u To prevent class extending and method overriding using the **final** modifier (§11.14).



Inheritance

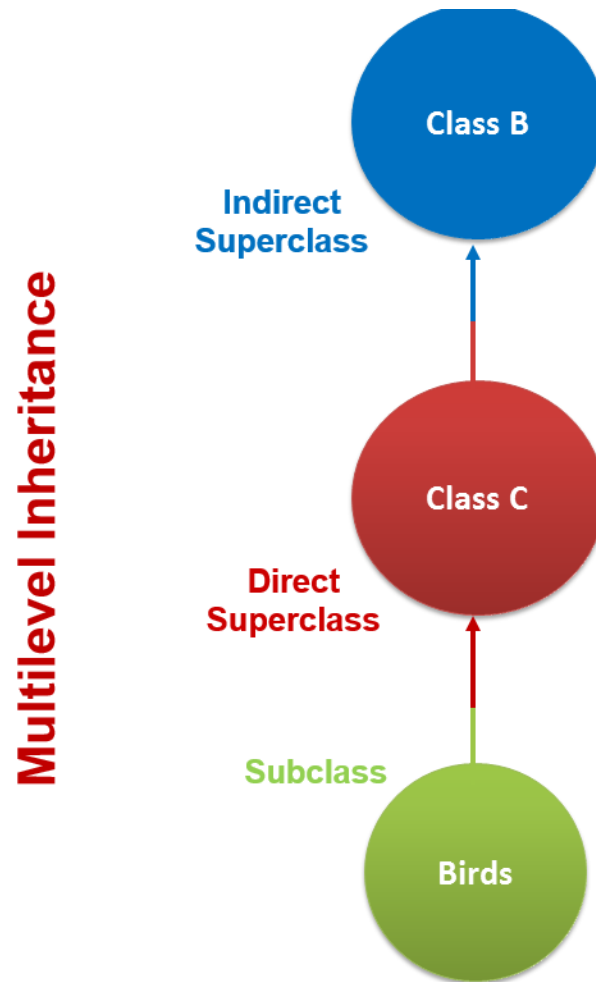


Superclasses and Subclasses

- Inheritance enables you to define a general class (i.e., a superclass) and later extend it to more specialized classes (i.e., subclasses).*



Multi-level Inheritance

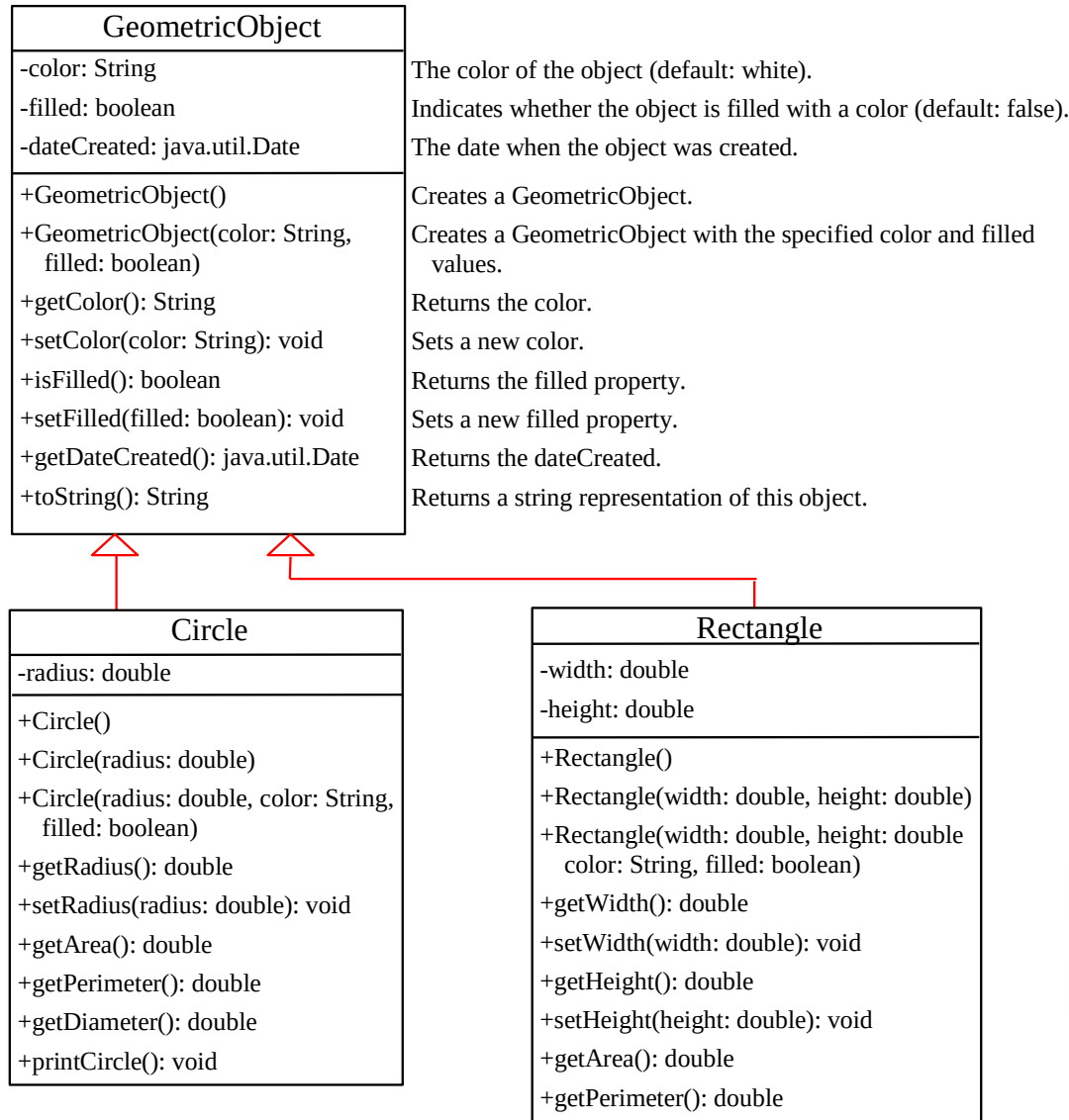


Example

- Geometric objects have many common properties and behaviors. They can be drawn in a certain color and be filled or unfilled. Thus a general class **GeometricObject** can be used to model all geometric objects such as Rectangle and Circle.



Superclasses and Subclasses



Superclasses and Subclasses

- In Java terminology, a class **C1** extended from another class **C2** is called a *subclass*, and **C2** is called a *superclass*. A superclass is also referred to as a *parent class* or a *base class*, and a subclass as a *child class*, an *extended class*, or a *derived class*. **A subclass inherits accessible data fields and methods from its superclass and may also add new data fields and methods.**
- The **Circle** class inherits all accessible data fields and methods from the **GeometricObject** class. In addition, it has a new data field, **radius**, and its associated getter and setter methods and various others such as **getArea()**.



Implementation

```
public class SimpleGeometricObject {
    private String color = "white";
    private boolean filled;
    private java.util.Date dateCreated;

    /** Construct a default geometric object */
    public SimpleGeometricObject() {
        dateCreated = new java.util.Date();
    }

    •

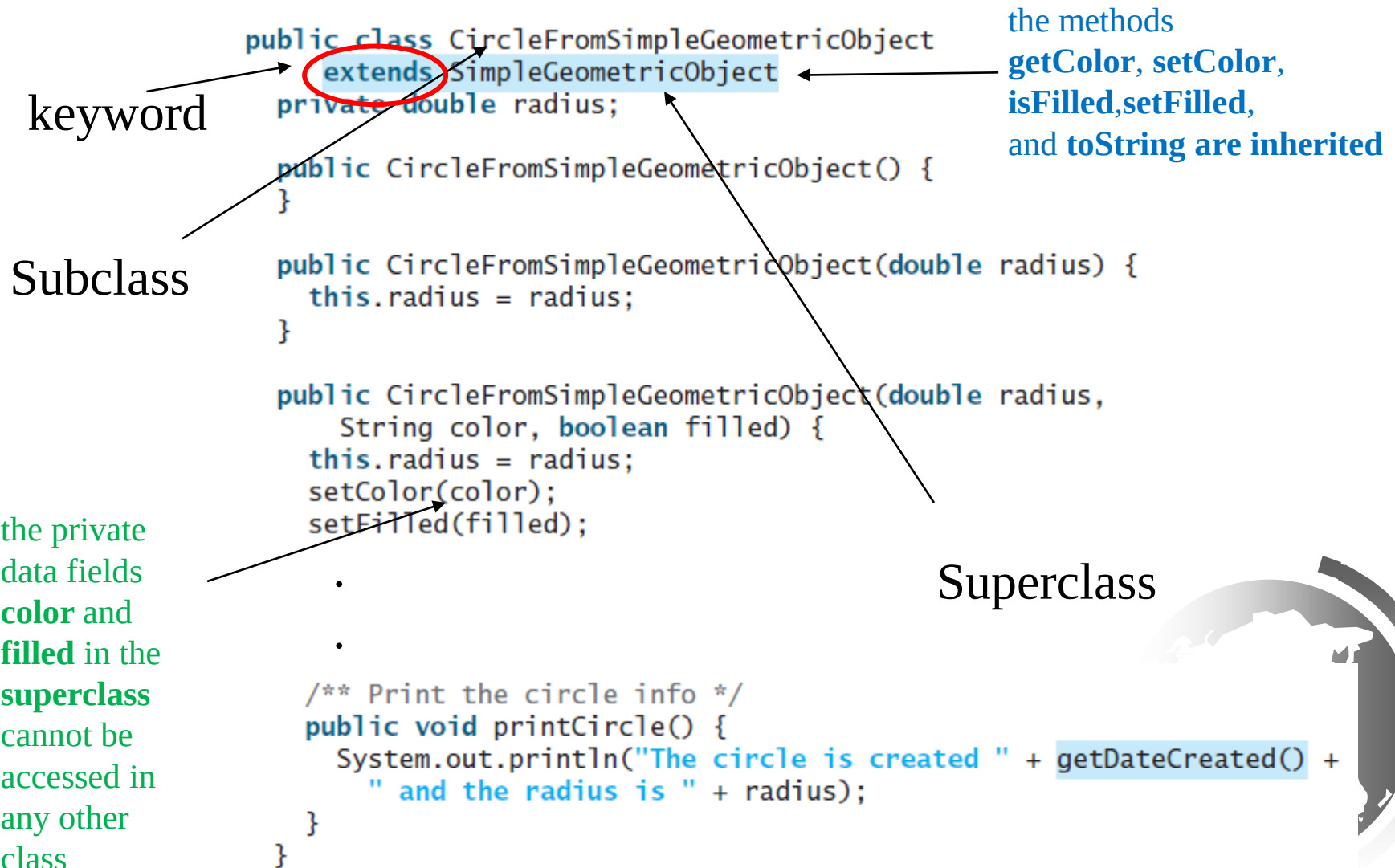
    •

    •

    /** Return a string representation of this object */
    public String toString() {
        return "created on " + dateCreated + "\ncolor: " + color +
            " and filled: " + filled;
    }
}
```



Implementation



Implementation

```
public class TestCircleRectangle {  
    public static void main(String[] args) {  
        CircleFromSimpleGeometricObject circle =  
            new CircleFromSimpleGeometricObject(1);  
        System.out.println("A circle " + circle.toString());  
        System.out.println("The color is " + circle.getColor());  
        System.out.println("The radius is " + circle.getRadius());  
        System.out.println("The area is " + circle.getArea());  
        System.out.println("The diameter is " + circle.getDiameter());  
  
        RectangleFromSimpleGeometricObject rectangle =  
            new RectangleFromSimpleGeometricObject(2, 4);  
        System.out.println("\nA rectangle " + rectangle.toString());  
        System.out.println("The area is " + rectangle.getArea());  
        System.out.println("The perimeter is " +  
            rectangle.getPerimeter());  
    }  
}
```

A method in superclass

A method in subclass



REMARKS

- A subclass is not a subset of its superclass. In fact, a subclass usually contains more information and methods than its superclass.
- Private data fields in a superclass are not accessible outside the class. Therefore, they cannot be used directly in a subclass. They can, however, be accessed/mutated through **public** accessors and mutators if defined in the superclass.
- Inheritance is used to model the is-a relationship. Do not blindly extend a class just for the sake of reusing methods. For example, it makes no sense for a **Tree** class to extend a **Person** class, even though they share common properties such as height and weight. **A subclass and its superclass must have the is-a relationship.**
- A Java class may inherit directly from only one superclass. This restriction is known as *single inheritance*.

Inheritance of Constructors

- Unlike properties and methods, the constructors of a superclass are not inherited by a subclass. They are invoked implicitly.
- They can also be explicitly invoked from the subclasses' constructors, using the keyword super. *If the keyword super is not explicitly used, the superclass's no-arg constructor is automatically invoked.*
- The syntax to call a superclass's constructor is **super()**, or **super(parameters)**, where the statement **super()** invokes the no-arg constructor of its superclass, and **super(arguments)** invokes the superclass constructor that matches the **arguments**.

Inheritance of Constructors

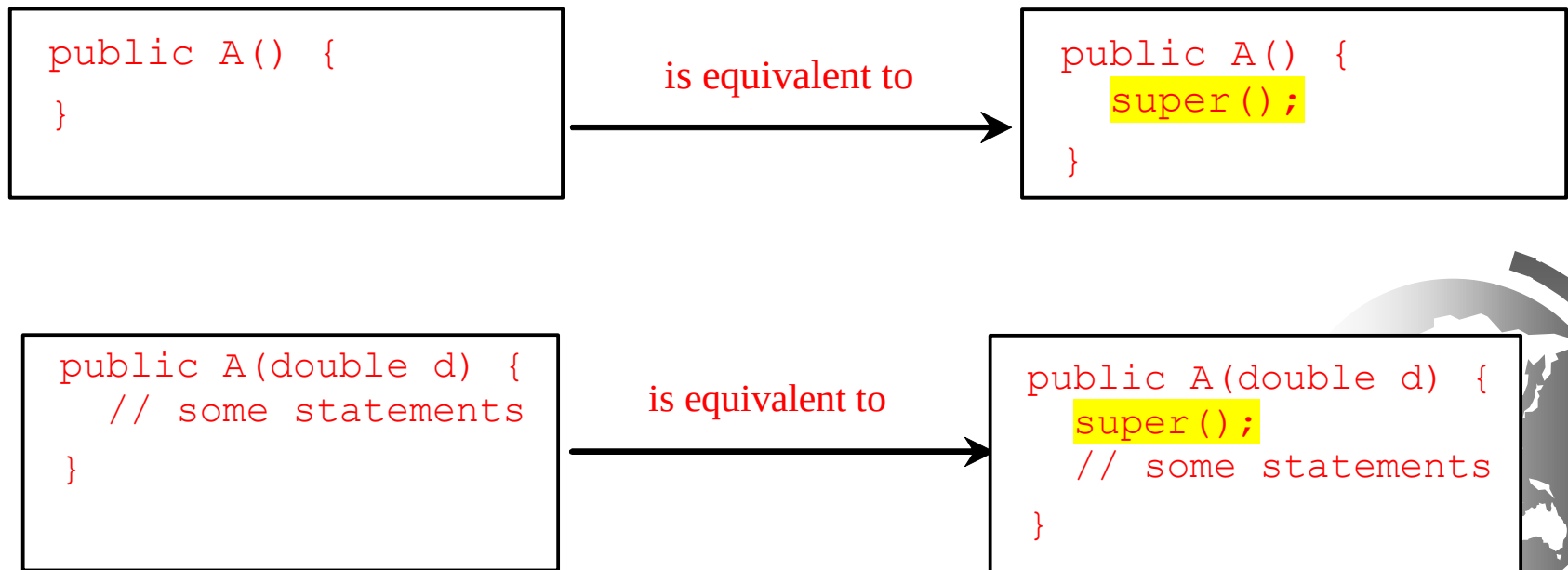
```
public CircleFromSimpleGeometricObject(  
    double radius, String color, boolean filled) {  
  
    super(color, filled);  
    this.radius = radius;  
}
```

Explicit call to a
particular constructor in
the superclass



Superclass's Constructor Is Always Invoked

A constructor may invoke an overloaded constructor or its superclass's constructor. If none of them is invoked explicitly, the compiler puts super() as **the first statement** in the constructor. For example,



Example

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Performs Faculty's tasks");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Performs Employee's tasks ");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Performs Person's tasks");  
    }  
}
```

- (1) Performs Person's tasks
- (2) Invoke Employee's overloaded constructor
- (3) Performs Employee's tasks
- (4) Performs Faculty's tasks



Trace Execution

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
}
```

1. Start from the
main method

```
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```



Trace Execution

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
}
```

2. Invoke Faculty
constructor

```
public Faculty() {  
    System.out.println("(4) Faculty's no-arg constructor is invoked");  
}  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```



Trace Execution

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
}
```

```
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}
```

3. Invoke Employee's no-arg constructor

```
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
}
```

```
    public Employee(String s) {  
        System.out.println(s);  
    }  
}
```

```
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```



Trace Execution

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
}
```

```
public Faculty() {  
    System.out.println("(4) Faculty's no-arg constructor is invoked");  
}
```

4. Invoke Employee(String)
constructor

```
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
}
```

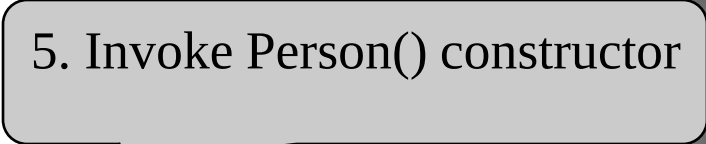
```
public Employee(String s) {  
    System.out.println(s);  
}
```

```
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```



Trace Execution

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```



5. Invoke Person() constructor

Trace Execution

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```



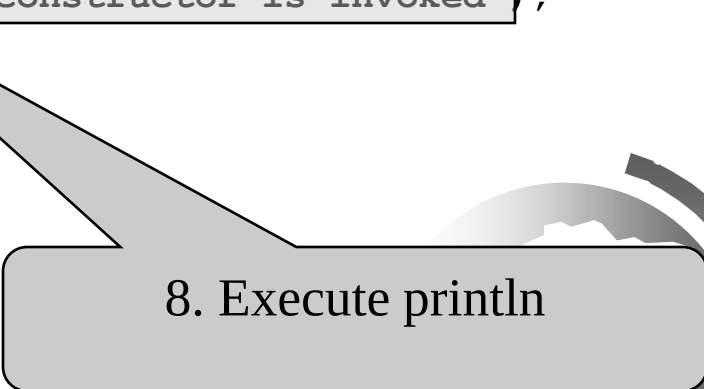
Trace Execution

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```

7. Execute println

Trace Execution

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```



8. Execute println

Trace Execution

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
}
```

```
public Faculty() {  
    System.out.println("(4) Faculty's no-arg constructor is invoked");  
}  
}
```

9. Execute println

```
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
}
```

```
public Employee(String s) {  
    System.out.println(s);  
}  
}
```

```
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```



CAUTION

- You must use the keyword super to call the superclass constructor. Invoking a superclass constructor's name in a subclass causes a syntax error.
- Java requires that the statement that uses the keyword super **appear first in the constructor.**
- The keyword super can also be used to call a method of the superclass as
`super.method(parameters)`



Example

```
class Base {
    int x;
    Base(int _x) {
        x = _x;
    }
}

class Derived extends Base {
    int y;
    Derived(int _x, int _y) {
        super(_x);
        y = _y;
    }
    void Display() {
        System.out.println("x = "+x+", y = "+y);
    }
}

public class Exam {
    public static void main(String[] args) {
        Derived d = new Derived(10, 20);
        d.Display();
    }
}
```



Problems @ Javadoc Declaration Console
<terminated> Exam [Java Application] C:\Program Files\
x = 10, y = 20

Example on the Impact of a Superclass without no-arg Constructor

If a class is designed to be extended, it is better to provide a no-arg constructor to avoid programming errors. Consider the following code:

```
1 public class Apple extends Fruit {  
2 }  
3  
4 class Fruit {  
5     public Fruit(String name) {  
6         System.out.println("Fruit's constructor is invoked");  
7     }  
8 }
```

Since no constructor is explicitly defined in **Apple**, **Apple**'s default no-arg constructor is defined implicitly. Since **Apple** is a subclass of **Fruit**, **Apple**'s default constructor automatically invokes **Fruit**'s no-arg constructor. However, **Fruit** does not have a no-arg constructor, because **Fruit** has an explicit constructor defined. Therefore, the program cannot be compiled.



Defining a Subclass

A subclass inherits from a superclass. You can also:

- Add new properties
- Add new methods
- **Override** the methods of the superclass



Overriding Methods in the Superclass

- A subclass inherits methods from a superclass. Sometimes it is necessary for the subclass to **modify the implementation** of a method defined in the superclass. This is referred to as *method overriding*.
- *To override a method, the method must be defined in the subclass using the same signature and the same return type as in its superclass.*

```
public class CircleFromSimpleGeometricObject
    extends SimpleGeometricObject {
    // Other methods are omitted

    // Override the toString method defined in the superclass
    public String toString() {
        return super.toString() + "\nradius is " + radius;
    }
}
```



REMARKS

- An instance method can be overridden only if it is accessible. **Thus a private method cannot be overridden**, because it is not accessible outside its own class. If a method defined in a subclass is private in its superclass, the two methods are completely unrelated.
- Like an instance method, a static method can be inherited. **However, a static method cannot be overridden**. If a static method defined in the superclass is redefined in a subclass, the method defined in the superclass is **hidden**.



Overriding vs. Overloading

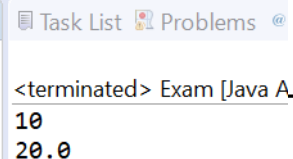
On the left, **p(double i)** in class **A** overrides the same method defined in class **B**. On the right, the class **A** has two overloaded methods: **p(double i)** and **p(int i)**. The method **p(double i)** is inherited from **B**.

```
public class Test {  
    public static void main(String[] args) {  
        A a = new A();  
        a.p(10);  
        a.p(10.0);  
    }  
}  
  
class B {  
    public void p(double i) {  
        System.out.println(i * 2);  
    }  
}  
  
class A extends B {  
    // This method overrides the method in B  
    public void p(double i) {  
        System.out.println(i);  
    }  
}
```



```
<terminated> Exam [Java Application] C:\Program Files\J  
10.0  
10.0
```

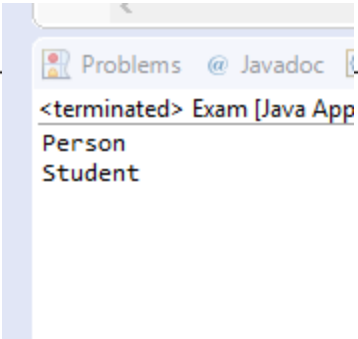
```
public class Test {  
    public static void main(String[] args) {  
        A a = new A();  
        a.p(10);  
        a.p(10.0);  
    }  
}  
  
class B {  
    public void p(double i) {  
        System.out.println(i * 2);  
    }  
}  
  
class A extends B {  
    // This method overloads the method in B  
    public void p(int i) {  
        System.out.println(i);  
    }  
}
```



```
<terminated> Exam [Java A]  
10  
20.0
```

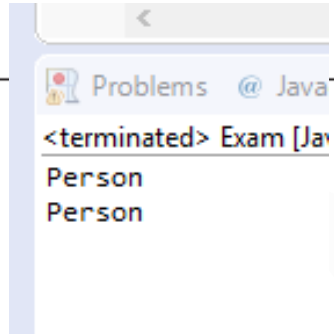
Examples

```
public class Test {  
    public static void main(String[] args) {  
        new Person().printPerson();  
        new Student().printPerson();  
    }  
}  
  
class Student extends Person {  
    public String getInfo() {  
        return "Student";  
    }  
}  
  
class Person {  
    public String getInfo() {  
        return "Person";  
    }  
  
    public void printPerson() {  
        System.out.println(getInfo());  
    }  
}
```



Problems @ Javadoc
<terminated> Exam [Java App]
Person
Student

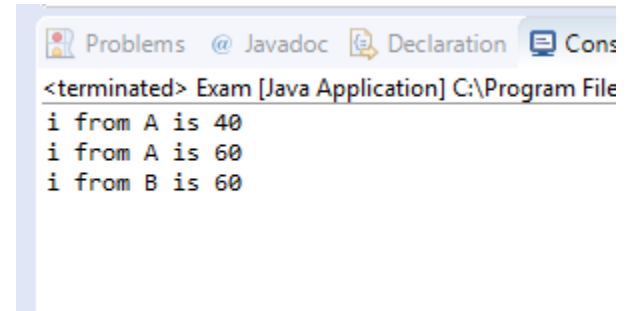
```
public class Test {  
    public static void main(String[] args) {  
        new Person().printPerson();  
        new Student().printPerson();  
    }  
}  
  
class Student extends Person {  
    private String getInfo() {  
        return "Student";  
    }  
}  
  
class Person {  
    private String getInfo() {  
        return "Person";  
    }  
  
    public void printPerson() {  
        System.out.println(getInfo());  
    }  
}
```



Problems @ Java
<terminated> Exam [Java App]
Person
Person

Examples

```
public class Test {  
    public static void main(String[] args) {  
        new A();  
        new B();  
    }  
}  
  
class A {  
    int i = 7;  
  
    public A() {  
        setI(20);  
        System.out.println("i from A is " + i);  
    }  
  
    public void setI(int i) {  
        this.i = 2 * i;  
    }  
}  
  
class B extends A {  
    public B() {  
        System.out.println("i from B is " + i);  
    }  
  
    public void setI(int i) {  
        this.i = 3 * i;  
    }  
}
```



The Object Class and Its Methods

- Every class in Java is descended from the `java.lang.Object` class. If no inheritance is specified when a class is defined, the superclass of the class is `Object`.

```
public class Circle {  
    ...  
}
```

Equivalent

```
public class Circle extends Object {  
    ...  
}
```

The Object Class and Its Methods

- All classes in Java inherit directly or indirectly from the Object class (package java.lang), so its 11 methods are inherited by all other classes.
- *The **Object class** is the parent class of all the classes in java by default.*



The Object Class and Its Methods

Method	Description
<code>public final Class getClass()</code>	returns the Class class object of this object. The Class class can further be used to get the metadata of this class.
<code>public int hashCode()</code>	returns the hashCode number for this object.
<code>public boolean equals(Object obj)</code>	compares the given object to this object.
<code>protected Object clone() throws CloneNotSupportedException</code>	creates and returns the exact copy (clone) of this object.
<code>public String toString()</code>	returns the string representation of this object.
<code>public final void notify()</code>	wakes up single thread, waiting on this object's monitor.
<code>public final void notifyAll()</code>	wakes up all the threads, waiting on this object's monitor.
<code>public final void wait(long timeout) throws InterruptedException</code>	causes the current thread to wait for the specified milliseconds, until another thread notifies (invokes <code>notify()</code> or <code>notifyAll()</code> method).
<code>public final void wait(long timeout, int nanos) throws InterruptedException</code>	causes the current thread to wait for the specified milliseconds and nanoseconds, until another thread notifies (invokes <code>notify()</code> or <code>notifyAll()</code> method).
<code>public final void wait() throws InterruptedException</code>	causes the current thread to wait, until another thread notifies (invokes <code>notify()</code> or <code>notifyAll()</code> method).
<code>protected void finalize() throws Throwable</code>	is invoked by the garbage collector before object is being garbage collected.

returns an integer that is unique for each instance

two objects are equal if and only if they are stored in the same memory address

The toString() method in Object

- The `toString()` method returns a string representation of the object. The default implementation returns a string consisting of a class name of which the object is an instance, the at sign (`@`), and a number representing this object.

```
Loan loan = new Loan();
```

```
System.out.println(loan.toString());
```

- The code displays something like `Loan@15037e53`. This message is not very helpful or informative. Usually you should override the `toString` method so that it returns a digestible string representation of the object.
- `System.out.println(loan.getClass())` displays `class Loan`

Example

```
class Animal{  
  void eat(){System.out.println("eating...");}  
}  
class Dog extends Animal{  
  void bark(){System.out.println("barking...");}  
}  
class TestInheritance{  
  public static void main(String args[]){  
    Dog d=new Dog();  
    d.bark();  
    d.eat();  
  }}
```

Output:

```
barking...  
eating...
```



Example

```
class Animal{
    void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
    void bark(){System.out.println("barking...");}
}
class BabyDog extends Dog{
    void weep(){System.out.println("weeping...");}
}
class TestInheritance2{
    public static void main(String args[]){
        BabyDog d=new BabyDog();
        d.weep();
        d.bark();
        d.eat();
    }}

```

Output:

```
weeping...
barking...
eating...
```



Example

```
class Teacher {  
    private String designation = "Teacher";  
    private String collegeName = "Beginnersbook";  
    public String getDesignation() {  
        return designation;  
    }  
    protected void setDesignation(String designation) {  
        this.designation = designation;  
    }  
    protected String getCollegeName() {  
        return collegeName;  
    }  
    protected void setCollegeName(String collegeName) {  
        this.collegeName = collegeName;  
    }  
    void does(){  
        System.out.println("Teaching");  
    }  
}
```

```
public class JavaExample extends Teacher{  
    String mainSubject = "Physics";  
    public static void main(String args[]){  
        JavaExample obj = new JavaExample();  
        /* Note: we are not accessing the data members  
        * directly we are using public getter method  
        * to access the private members of parent class  
        */  
        System.out.println(obj.getCollegeName());  
        System.out.println(obj.getDesignation());  
        System.out.println(obj.mainSubject);  
        obj.does();  
    }  
}
```

The output is:

```
Beginnersbook  
Teacher  
Physics  
Teaching
```



Example

```
class ParentClass{
    //Parent class constructor
    ParentClass(){
        System.out.println("Constructor of Parent");
    }
    void disp(){
        System.out.println("Parent Method");
    }
}
class JavaExample extends ParentClass{
    JavaExample(){
        System.out.println("Constructor of Child");
    }
    void disp(){
        System.out.println("Child Method");
        //Calling the disp() method of parent class
        super.disp();
    }
    public static void main(String args[]){
        //Creating the object of child class
        JavaExample obj = new JavaExample();
        obj.disp();
    }
}
```

The output is :

```
Constructor of Parent
Constructor of Child
Child Method
Parent Method
```



Example

```
class Animal {
    public Animal() {
        System.out.println("I am an Animal");
    }

    public void eat() {
        System.out.println("I can eat");
    }
}

class Dog extends Animal {
    public Dog(){
        super();
        System.out.println("I am a dog");
    }

    @Override
    public void eat() {
        super.eat();
        System.out.println("I eat dog food");
    }

    public void bark() {
        System.out.println("I can bark");
    }
}
```

```
class Main {
    public static void main(String[] args) {
        Dog dog1 = new Dog();

        dog1.eat();
        dog1.bark();
    }
}
```

Output

```
I am an Animal
I am a dog
I can eat
I eat dog food
I can bark
```



Example

```
public class Point {  
    // fields marking X and Y position of the point  
    public int x;  
    public int y;  
  
    // one constructor  
    public Point(int x, int y) {  
        super();  
        this.x = x;  
        this.y = y;  
    }  
  
    // getter and setter methods  
    public int getX() {  
        return x;  
    }  
  
    public void setX(int x) {  
        this.x = x;  
    }  
  
    public int getY() {  
        return y;  
    }  
  
    public void setY(int y) {  
        this.y = y;  
    }  
}
```

```
public class ColoredPoint extends Point {  
  
    // new field added to store the color name  
    public String colorName;  
  
    public ColoredPoint(int x, int y, String colorName) {  
        super(x, y);  
        this.colorName = colorName;  
    }  
  
    public String getColorName() {  
        return colorName;  
    }  
  
    public void setColorName(String colorName) {  
        this.colorName = colorName;  
    }  
}
```



Example

```
public class Animal {
    public Animal() {
        System.out.println("A new animal has been created!");
    }

    public void sleep() {
        System.out.println("An animal sleeps...");
    }

    public void eat() {
        System.out.println("An animal eats...");
    }
}

public class Bird extends Animal {
    public Bird() {
        super();
        System.out.println("A new bird has been created!");
    }

    @Override
    public void sleep() {
        System.out.println("A bird sleeps...");
    }

    @Override
    public void eat() {
        System.out.println("A bird eats...");
    }
}

public class Dog extends Animal {
    public Dog() {
        super();
        System.out.println("A new dog has been created!");
    }

    @Override
    public void sleep() {
        System.out.println("A dog sleeps...");
    }

    @Override
    public void eat() {
        System.out.println("A dog eats...");
    }
}
```

```
public class MainClass {
    public static void main(String[] args) {
        Animal animal = new Animal();
        Bird bird = new Bird();
        Dog dog = new Dog();

        System.out.println();

        animal.sleep();
        animal.eat();

        bird.sleep();
        bird.eat();

        dog.sleep();
        dog.eat();
    }
}
```

```
A new animal has been created!
A new animal has been created!
A new bird has been created!
A new animal has been created!
A new dog has been created!

An animal sleeps...
An animal eats...
A bird sleeps...
A bird eats...
A dog sleeps...
A dog eats...
```

Polymorphism

- A class defines a type. A type defined by a subclass is called a *subtype*, and a type defined by its superclass is called a *supertype*. Therefore, you can say that **Circle** is a subtype of **GeometricObject** and **GeometricObject** is a supertype for **Circle**.

Polymorphism means that a variable of a supertype can refer to a subtype object.



Polymorphism

- Suppose we create a program that simulates the **movement** of several types of **animals** for a biological study.
- Classes **Fish**, **Frog** and **Bird** represent the three types of animals under investigation.
- Imagine that each of these classes **extends superclass **Animal****, which contains a method **move**.
- Each subclass implements its method **move**.
- We can maintain an **array of **Animal** references to objects** of the various ****Animal** subclasses**.
- To simulate the animals' movements, we can send each object the same **message** once per second—namely, **move**.



Polymorphism

- However, each specific type of Animal responds to a move message in a **unique way**—a Fish might swim three feet, a Frog might jump five feet and a Bird might fly ten feet.
- Although we issue the **same message** (i.e., move) to each animal object generically, each object knows how to modify its x-y coordinates appropriately for its specific type of movement.
- Relying on each object to know how to “do the right thing” in response to the same method call is the **key concept of polymorphism**.

The same message sent to a variety of objects has
“many forms” of results—hence the term

Polymorphism



Polymorphism, Dynamic Binding and Generic Programming

```
public class PolymorphismDemo {  
    public static void main(String[] args) {  
        m(new GraduateStudent());  
        m(new Student());  
        m(new Person());  
        m(new Object());  
    }  
}
```

```
public static void m(Object x) {  
    System.out.println(x.toString());  
}
```

```
class GraduateStudent extends Student {  
}
```

```
class Student extends Person {  
    public String toString() {  
        return "Student";  
    }  
}
```

```
class Person extends Object {  
    public String toString() {  
        return "Person";  
    }  
}
```

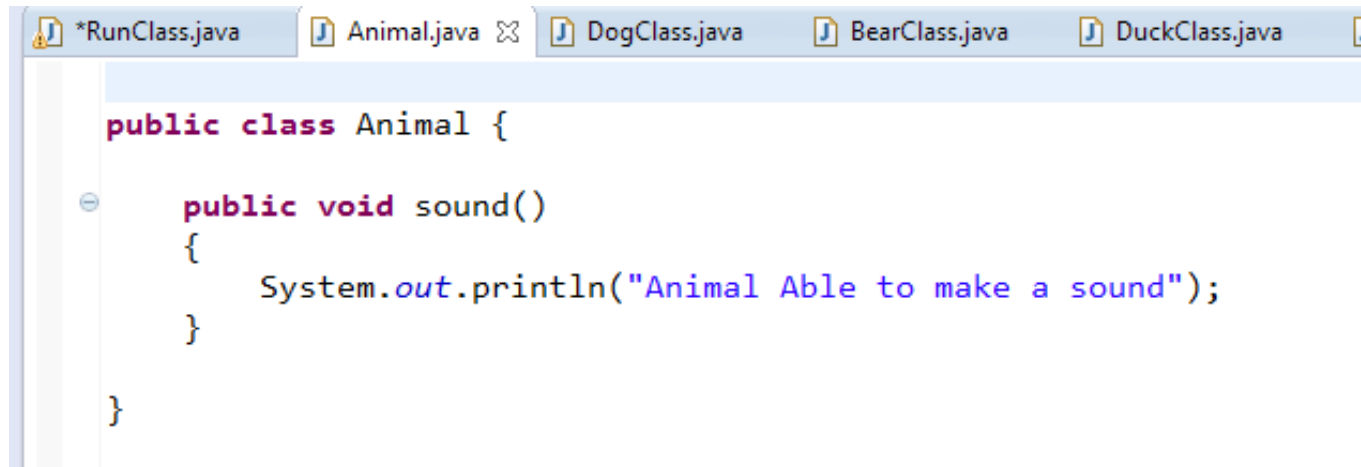
```
Student  
Student  
Person  
java.lang.Object@5e481248
```

Method **m** takes a parameter of the Object type. You can invoke it with any object.

polymorphism occurs when a program invokes a method through a superclass variable. At execution time, the correct subclass version of the method is called, based on the type of the reference stored in the superclass variable.

- When the method `m(Object x)` is executed, the argument `x`'s `toString` method is invoked. `x` may be an instance of `GraduateStudent`, `Student`, `Person`, or `Object`.
- Classes `GraduateStudent`, `Student`, `Person`, and `Object` have their own implementation of the `toString` method. Which implementation is used will be determined dynamically by the Java Virtual Machine at runtime. This capability is known as *dynamic binding*.

Example

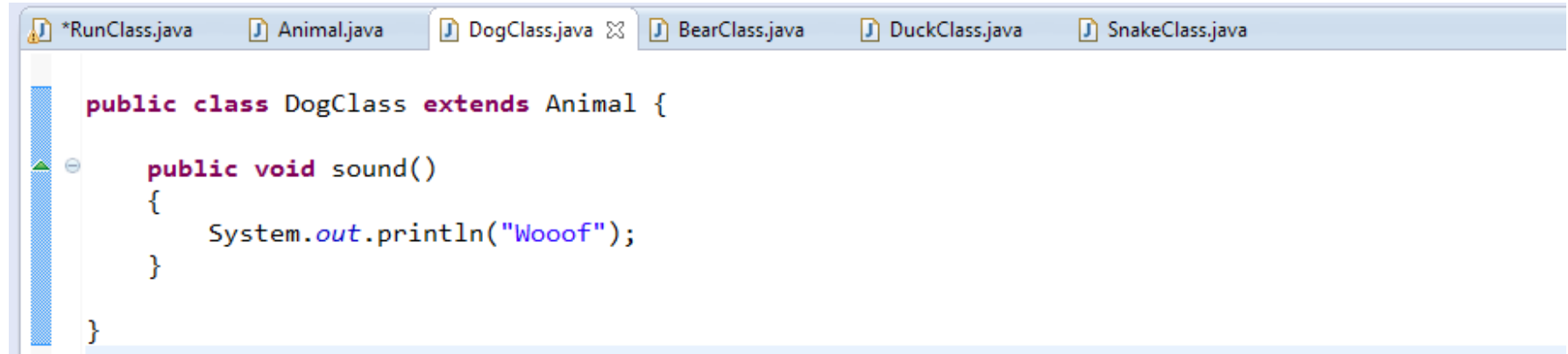


The screenshot shows a Java IDE with several tabs at the top: *RunClass.java, Animal.java, DogClass.java, BearClass.java, and DuckClass.java. The Animal.java tab is active, displaying the following code:

```
public class Animal {  
    public void sound()  
    {  
        System.out.println("Animal Able to make a sound");  
    }  
}
```

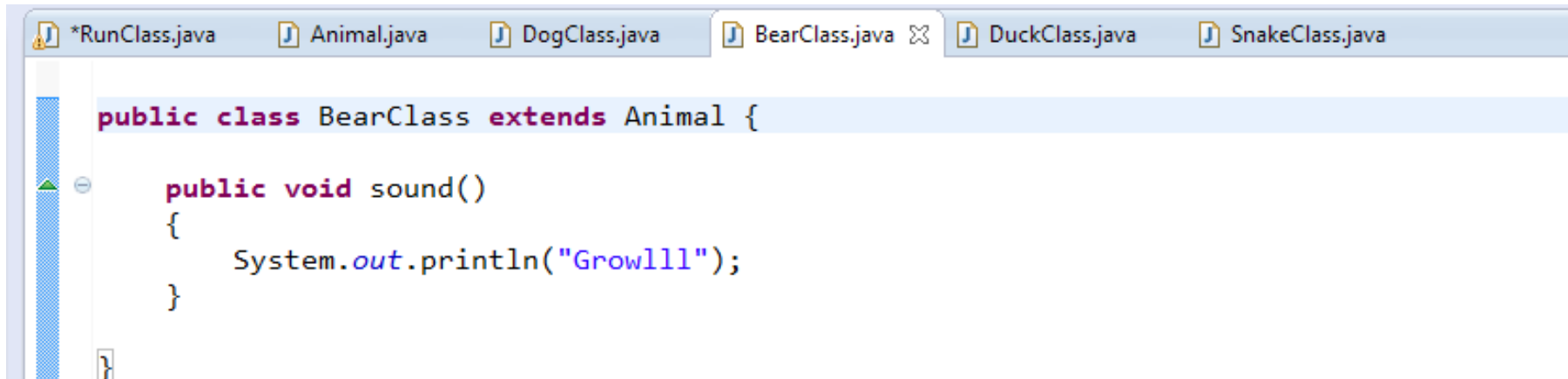


Example



A screenshot of an IDE window showing the `DogClass.java` file. The window has several tabs at the top: `*RunClass.java`, `Animal.java`, `DogClass.java` (which is the active tab), `BearClass.java`, `DuckClass.java`, and `SnakeClass.java`. The code in the editor is as follows:

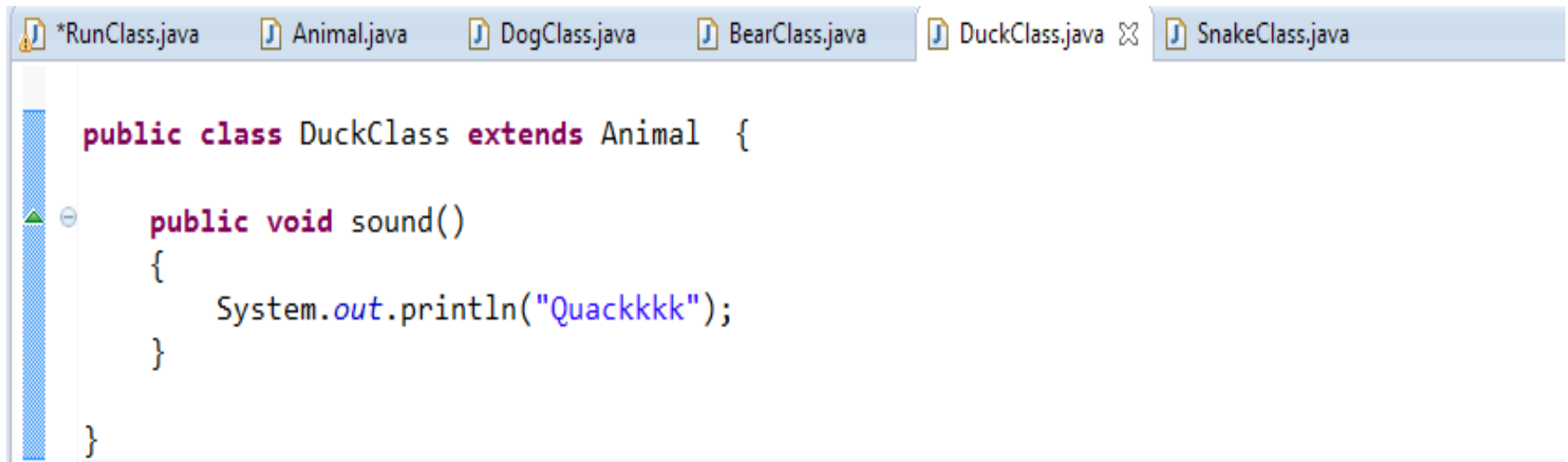
```
public class DogClass extends Animal {  
    public void sound()  
    {  
        System.out.println("Woof");  
    }  
}
```



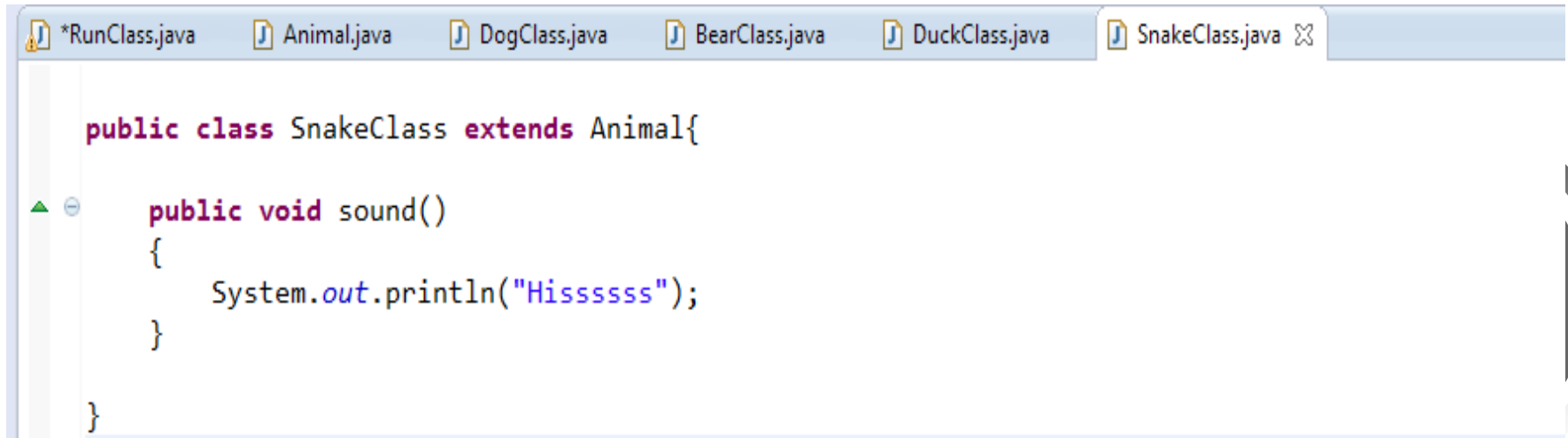
A screenshot of an IDE window showing the `BearClass.java` file. The window has the same tabs as the previous screenshot: `*RunClass.java`, `Animal.java`, `DogClass.java`, `BearClass.java` (which is the active tab), `DuckClass.java`, and `SnakeClass.java`. The code in the editor is as follows:

```
public class BearClass extends Animal {  
    public void sound()  
    {  
        System.out.println("Growl");  
    }  
}
```

Example

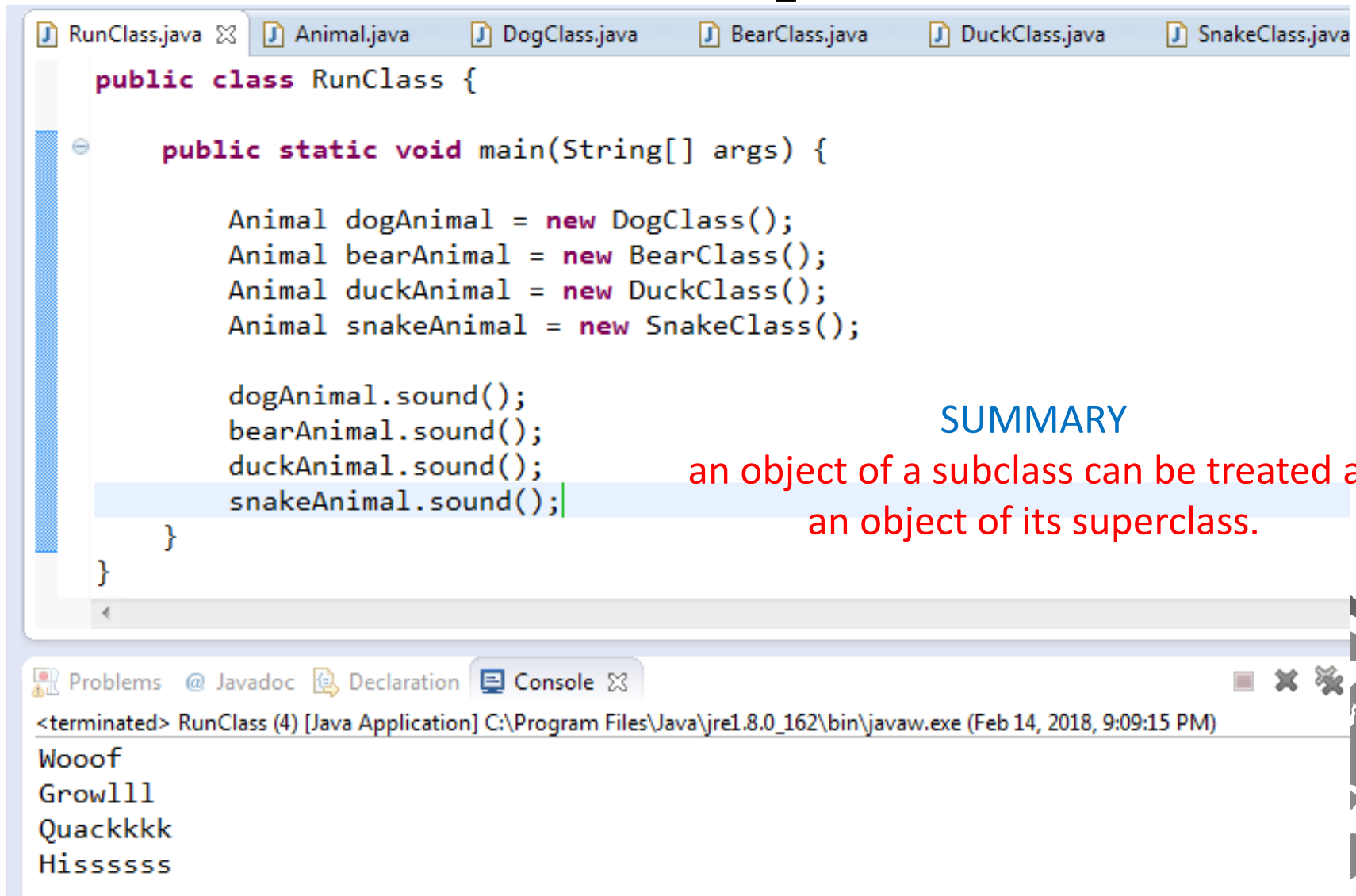


```
public class DuckClass extends Animal {  
    public void sound()  
    {  
        System.out.println("Quackkkk");  
    }  
}
```



```
public class SnakeClass extends Animal{  
    public void sound()  
    {  
        System.out.println("Hissssss");  
    }  
}
```

Example



The screenshot shows an IDE with several tabs: RunClass.java, Animal.java, DogClass.java, BearClass.java, DuckClass.java, and SnakeClass.java. The RunClass.java tab is active, displaying the following code:

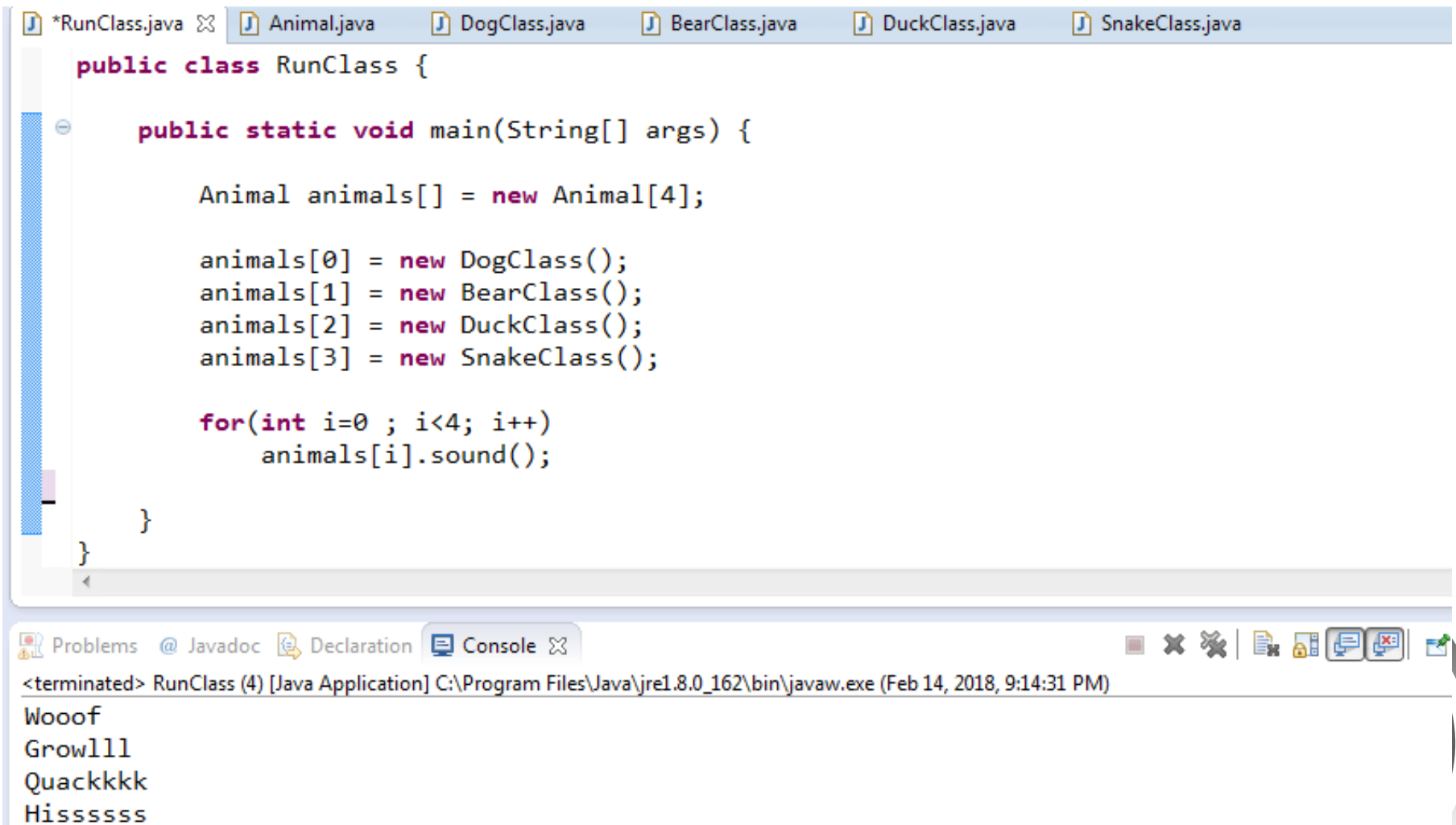
```
public class RunClass {  
  
    public static void main(String[] args) {  
  
        Animal dogAnimal = new DogClass();  
        Animal bearAnimal = new BearClass();  
        Animal duckAnimal = new DuckClass();  
        Animal snakeAnimal = new SnakeClass();  
  
        dogAnimal.sound();  
        bearAnimal.sound();  
        duckAnimal.sound();  
        snakeAnimal.sound();  
    }  
}
```

The line `snakeAnimal.sound();` is highlighted in blue. To the right of the code, the text "SUMMARY" is written in blue, followed by "an object of a subclass can be treated as an object of its superclass." in red.

At the bottom, the Console tab is active, showing the output of the program:

```
<terminated> RunClass (4) [Java Application] C:\Program Files\Java\jre1.8.0_162\bin\javaw.exe (Feb 14, 2018, 9:09:15 PM)  
Woof  
Growlll  
Quackkkk  
Hissssss
```

Example



The screenshot shows an IDE with a tab bar at the top containing: *RunClass.java, Animal.java, DogClass.java, BearClass.java, DuckClass.java, and SnakeClass.java. The main editor displays the following Java code:

```
public class RunClass {  
    public static void main(String[] args) {  
        Animal animals[] = new Animal[4];  
  
        animals[0] = new DogClass();  
        animals[1] = new BearClass();  
        animals[2] = new DuckClass();  
        animals[3] = new SnakeClass();  
  
        for(int i=0 ; i<4; i++)  
            animals[i].sound();  
    }  
}
```

At the bottom, the 'Console' tab is active, showing the output of the program:

```
<terminated> RunClass (4) [Java Application] C:\Program Files\Java\jre1.8.0_162\bin\javaw.exe (Feb 14, 2018, 9:14:31 PM)  
Woof  
Growlll  
Quackkkk  
Hissssss
```

Summary

- Polymorphism occurs when a program invokes a method through a superclass variable
- At execution time, the correct subclass version of the method is called, based on the type of the reference stored in the superclass variable.



Example

```
public class Exam{
    public static void main(String[] args) {
        Cat myCat = new Cat();
        Animal myAnimal = myCat;
        Animal.testClassMethod();
        myAnimal.testInstanceMethod();
    }
}
class Animal {
    public static void testClassMethod() {
        System.out.println("The static method in Animal");
    }
    public void testInstanceMethod() {
        System.out.println("The instance method in Animal");
    }
}
class Cat extends Animal {
    public static void testClassMethod() {
        System.out.println("The static method in Cat");
    }
    public void testInstanceMethod() {
        System.out.println("The instance method in Cat");
    }
}
```



Problems @ Javadoc Declaration Console

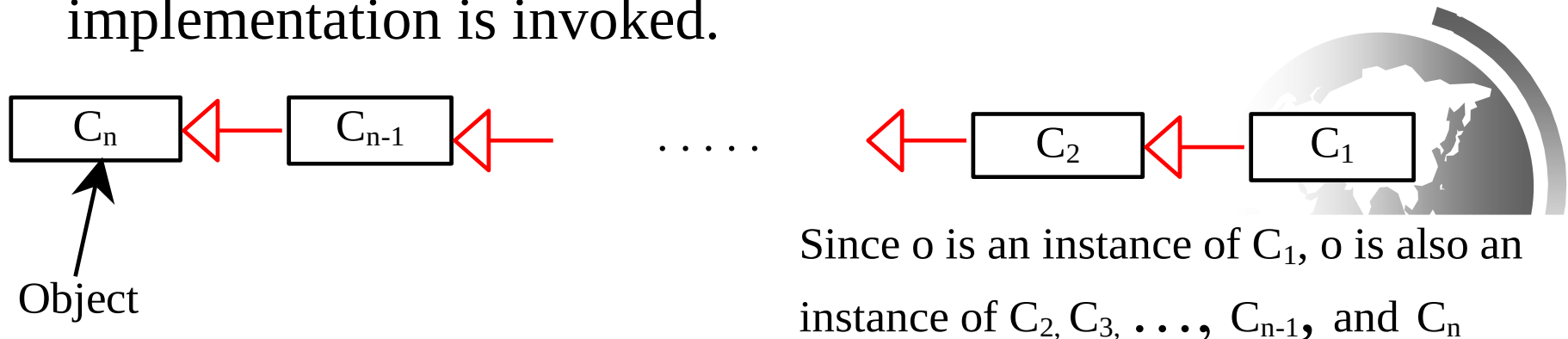
<terminated> Exam [Java Application] C:\Program Files\Java\jre1.8

The static method in Animal

The instance method in Cat

Dynamic Binding

- Dynamic binding works as follows: Suppose an object **o** is an instance of C_1 . In fact, it is also an instance of $C_2, \dots, C_{n-1}$, and C_n , where C_1 is a subclass of C_2 , C_2 is a subclass of $C_3, \dots$, and C_{n-1} is a subclass of C_n . That is, C_n is the most general class, and C_1 is the most specific class.
- In Java, C_n is the **Object** class. If **o** invokes a method **p**, the JVM searches the implementation for the method **p** in $C_1, C_2, \dots, C_{n-1}$ and C_n , in this order, until it is found. Once an implementation is found, the search stops and the first-found implementation is invoked.



Method Matching vs. Binding

- ✦ Matching a method signature and binding a method implementation are two issues. The compiler finds a matching method according to parameter type, number of parameters, and order of the parameters at compilation time.
- ✦ A method may be implemented in several subclasses. The Java Virtual Machine dynamically binds the implementation of the method at runtime.



Generic Programming

```
public class PolymorphismDemo {
    public static void main(String[] args) {
        m(new GraduateStudent());
        m(new Student());
        m(new Person());
        m(new Object());
    }

    public static void m(Object x) {
        System.out.println(x.toString());
    }
}

class GraduateStudent extends Student {
}

class Student extends Person {
    public String toString() {
        return "Student";
    }
}

class Person extends Object {
    public String toString() {
        return "Person";
    }
}
```

Polymorphism allows methods to be used generically for a wide range of object arguments. This is known as generic programming. If a method's parameter type is a superclass (e.g., `Object`), you may pass an object to this method of any of the parameter's subclasses (e.g., `Student` or `String`). When an object (e.g., a `Student` object or a `String` object) is used in the method, the particular implementation of the method of the object that is invoked (e.g., `toString`) is determined dynamically.

Example

```
public class Exam {
    public static void main(String args[]){
        X x= new X();
        Y y = new Y();
        y.m2();
        x.m1();
        y.m1();
        x = y; // parent pointing to object of child
        x.m1() ;
        //y.a=10;
    }
}

class X{
    private int a;
    int b;
    public void m1(){
        System.out.println("This is method m1 of class X");
    }
}

class Y extends X{
    int c; // new instance variable of class Y
    public void m1(){
        // overridden method
        System.out.println("This is method m1 of class Y");
    }
    public void m2(){
        super.m1();
        System.out.println("This is method m2 of class Y");
    }
}
```



Problems @ Javadoc Declaration Console

<terminated> Exam [Java Application] C:\Program Files\Java\jre1.8.0_191\bin\java.exe

This is method m1 of class X
This is method m2 of class Y
This is method m1 of class X
This is method m1 of class Y
This is method m1 of class Y

Example

```
public class Exam{
    public static void main(String args[])
    {
        Vehicle v=new Bike();

        v.move();

        Vehicle v1=new Vehicle();

        v1.move();

    }
}
class Vehicle
{
    public void move()
    {
        System.out.println("vehicles can move");
    }
}
class Bike extends Vehicle
{
    public void move()
    {
        System.out.println("bike can move and accelerate too");
    }
}
```



Problems @ Javadoc Declaration Console X

<terminated> Exam [Java Application] C:\Program Files\Java\jre1.8.0_191\bin\java.exe
bike can move and accelerate too
vehicles can move

Casting Objects

- You have already used the casting operator to convert variables of one primitive type to another. *Casting* can also be used to convert an object of one class type to another within an inheritance hierarchy. In the preceding section, the statement
`m(new Student());`
assigns the object `new Student()` to a parameter of the `Object` type.

This statement is equivalent to:

```
Object o = new Student(); // Implicit casting, upcasting  
m(o);
```

The statement `Object o = new Student();`, known as implicit casting, is legal because an instance of `Student` is automatically an instance of `Object`.



Why Casting Is Necessary?

- F Suppose you want to assign the object reference `o` to a variable of the `Student` type using the following statement:

```
Student b = o;
```

A compile error would occur.

- F Why does the statement **Object o = new Student()** work and the statement **Student b = o** doesn't? This is because a `Student` object is always an instance of `Object`, but an `Object` is not necessarily an instance of `Student`. Even though you can see that `o` is really a `Student` object, the compiler is not so clever to know it. To tell the compiler that `o` is a `Student` object, use an explicit casting as follows:

```
Student b = (Student) o; // Explicit casting
```



Casting from Superclass to Subclass (Downcasting)

- Explicit casting must be used when casting an object from a superclass to a subclass.

```
Apple x = (Apple) fruit;
```

```
Orange x = (Orange) fruit;
```

- This type of casting may not always succeed. For the casting to be successful, **you must make sure that the superclass object to be downcasted is an instance of the subclass**. If the superclass object is not an instance of the subclass, a runtime ***ClassCastException*** occurs



Example

```
1 class Super {
2     void Sample() {
3         System.out.println("method of super class");
4     }
5 }
6
7 class Sub extends Super {
8     void Sample() {
9         System.out.println("method of sub class");
10    }
11 }
12 public class Exam{
13     public static void main(String args[]) {
14         Super obj = new Sub(); obj.Sample();
15     }
16 }
```

<terminated> Exam [Java Application] C:\Program Files\Java\jre1.8.0_191\bin\javaw.exe
method of sub class



Example

```
1 class Super {
2     void Sample() {
3         System.out.println("method of super class");
4     }
5 }
6
7 class Sub extends Super {
8     void Sample() {
9         System.out.println("method of sub class");
10    }
11 }
12 public class Exam{
13     public static void main(String args[]) {
14         Super obj = new Sub();
15         Sub sub = (Sub) obj; sub.Sample();
16     }
17 }
18
```

Problems @ Javadoc Declaration Console X

<terminated> Exam [Java Application] C:\Program Files\Java\jre1.8.0_191\bin\method of sub class



Example

```
1 class Flower
2 {
3     public void smell()
4     {
5         System.out.println("All flowers give smell");
6     }
7 }
8 class Rose extends Flower
9 {
10    public void smell() // II
11    {
12        System.out.println("Rose gives rosy smell");
13    }
14 }
15 public class Exam{
16    public static void main(String args[])
17    {
18        Flower f = new Flower();
19        Rose r = new Rose();
20
21        f.smell();
22        r.smell();
23
24        f = r;
25        f.smell();
26
27        //r = f; is not valid!!!
28        r = (Rose) f;
29        r.smell();
30    }
31 }
32
```

<

Problems @ Javadoc Declaration Console

<terminated> Exam [Java Application] C:\Program Files\Java\jre1.8.0_191\bin
All flowers give smell
Rose gives rosy smell
Rose gives rosy smell
Rose gives rosy smell



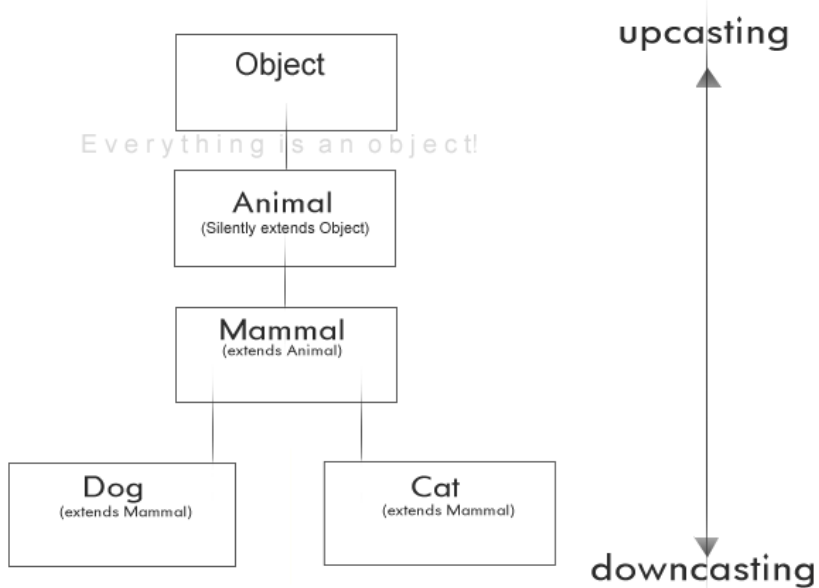
Example

```
1 |
2 class SuperClass {
3     void Sample() {
4         System.out.println("method of super class");
5     }
6 }
7
8 class SubClass extends SuperClass {
9     void Sample() {
10        System.out.println("method of sub class");
11    }
12 }
13 public class Exam{
14     public static void main(String args[]) {
15         SuperClass objOne = new SuperClass();
16         SubClass objTwo = new SubClass();
17
18         objTwo = (SubClass) objOne;
19         objTwo.Sample();
20     }
21 }
22
```

Problems @ Javadoc Declaration Console

<terminated> Exam [Java Application] C:\Program Files\Java\jre1.8.0_191\bin\javaw.exe (Nov 27, 2018, 10:25:13 AM)
Exception in thread "main" [java.lang.ClassCastException](#): SuperClass cannot be cast to SubClass
at Exam.main([Exam.java:18](#))

Example



```
class Animal {
    int health = 100;
}

class Mammal extends Animal { }

class Cat extends Mammal { }

class Dog extends Mammal { }

public class Test {
    public static void main(String[] args) {
        Cat c = new Cat();
        System.out.println(c.health);
        Dog d = new Dog();
        System.out.println(d.health);
    }
}
```

Example

- Casting does not change the object but labels in a different way. In particular, if you upcast a Cat to Animal, then the object is still a Cat. **However, it's Cat properties are hidden.** It gains its Cat properties back after being downcasted to Cat.

```
Cat c = new Cat();  
System.out.println(c);  
Mammal m = c; // upcasting  
System.out.println(m);
```

```
/*  
This printed:  
    Cat@a90653  
    Cat@a90653  
*/
```



Example

```
1 class Animal {
2     int health = 100;
3 }
4
5 class Mammal extends Animal { }
6
7 class Cat extends Mammal {
8     public int catHealth = 25;
9 }
10
11 class Dog extends Mammal { }
12
13 public class Exam {
14     public static void main(String[] args) {
15         Cat c = new Cat();
16         System.out.println(c);
17         Mammal m = c; // upcasting
18         System.out.println(m);
19
20         //hidden cat properties
21         System.out.println(c.catHealth);
22         System.out.println(m.catHealth);
23     }
24 }
25
```

ERROR!



Example

```
1 class Animal {
2     int health = 1;
3 }
4
5 class Mammal extends Animal {
6     int health = 2;
7 }
8
9 class Cat extends Mammal {
10     int health = 3;
11 }
12
13 class Dog extends Mammal {
14     int health = 4;
15 }
16
17 public class Exam {
18     public static void main(String[] args) {
19         Cat c = new Cat();
20         Mammal m = c; // upcasting
21         Animal a = m;
22
23
24         System.out.println(c.health);
25         System.out.println(m.health);
26         System.out.println(a.health);
27     }
28 }
29
```

Problems @ Javadoc Declaration Console

<terminated> Exam [Java Application] C:\Program Files\Java\jre1.8.0_191\bin\java

3
2
1



TIP

To help understand casting, you may also consider the analogy of fruit, apple, and orange with the Fruit class as the superclass for Apple and Orange.

- An apple is a fruit, so you can always safely assign an instance of Apple to a variable for Fruit.
- A fruit is not necessarily an apple, so you have to use explicit casting to assign an instance of Fruit to a variable of Apple.



The instanceof Operator

Use the instanceof operator to test whether an object is an instance of a class:

```
Object myObject = new Circle();  
... // Some lines of code  
/* Perform casting if myObject is an instance of Circle */  
if (myObject instanceof Circle) {  
    System.out.println("The circle diameter is " +  
        ((Circle)myObject).getDiameter());  
    ...  
}
```



The equals Method

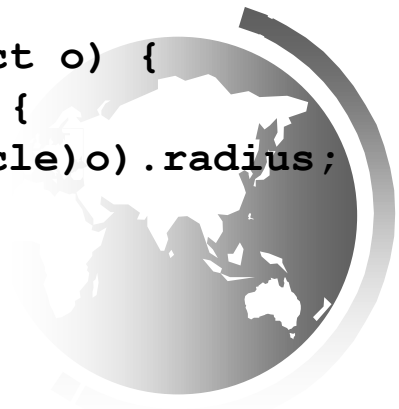
- Another method defined in the **Object** class that is often used is the **equals** method. This method compares two objects. The default implementation of the equals method in the Object class is as follows:

```
public boolean equals(Object obj) {  
    return this == obj;  
}
```

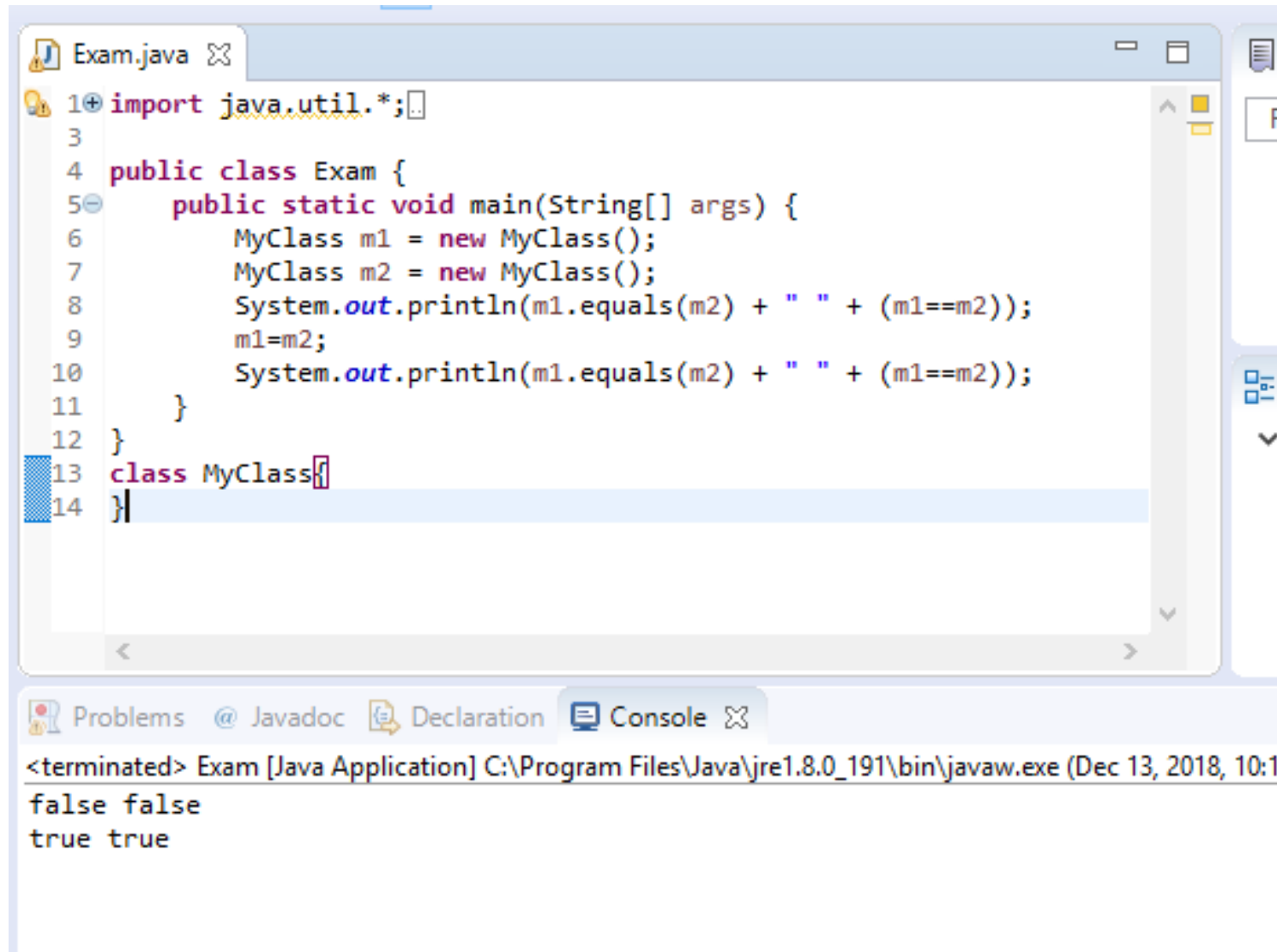
This implementation checks whether two reference variables point to the same object using the == operator.

The equals method can be overridden in the Circle class to make the test on radius values.

```
public boolean equals(Object o) {  
    if (o instanceof Circle) {  
        return radius == ((Circle)o).radius;  
    }  
    else  
        return false;  
}
```



The equals Method



The screenshot shows an IDE window titled 'Exam.java'. The code defines a 'MyClass' and a main method in the 'Exam' class. The main method creates two 'MyClass' objects, 'm1' and 'm2', and prints the results of 'm1.equals(m2)' and 'm1==m2' twice. The first time, both return 'false'. The second time, after 'm1=m2' is executed, both return 'true'. The console output at the bottom confirms this: 'false false' followed by 'true true'.

```
1 import java.util.*;
3
4 public class Exam {
5     public static void main(String[] args) {
6         MyClass m1 = new MyClass();
7         MyClass m2 = new MyClass();
8         System.out.println(m1.equals(m2) + " " + (m1==m2));
9         m1=m2;
10        System.out.println(m1.equals(m2) + " " + (m1==m2));
11    }
12 }
13 class MyClass{
14 }
```

Problems @ Javadoc Declaration Console

<terminated> Exam [Java Application] C:\Program Files\Java\jre1.8.0_191\bin\javaw.exe (Dec 13, 2018, 10:1
false false
true true

NOTE

- The == comparison operator is used for comparing two primitive data type values or for determining whether **two objects have the same references**.
- If the equals() method is intended to test whether two objects have the same contents, the method must be overridden in the defining class of the objects.



The protected Modifier

- ❑ The `protected` modifier can be applied on data and methods in a class. A protected data or a protected method in a public class can be accessed by any class in the same package or its subclasses, even if the subclasses are in a different package.
- ❑ `private`, `default`, `protected`, `public`

Visibility increases
→
`private`, `none` (if no modifier is used), `protected`, `public`

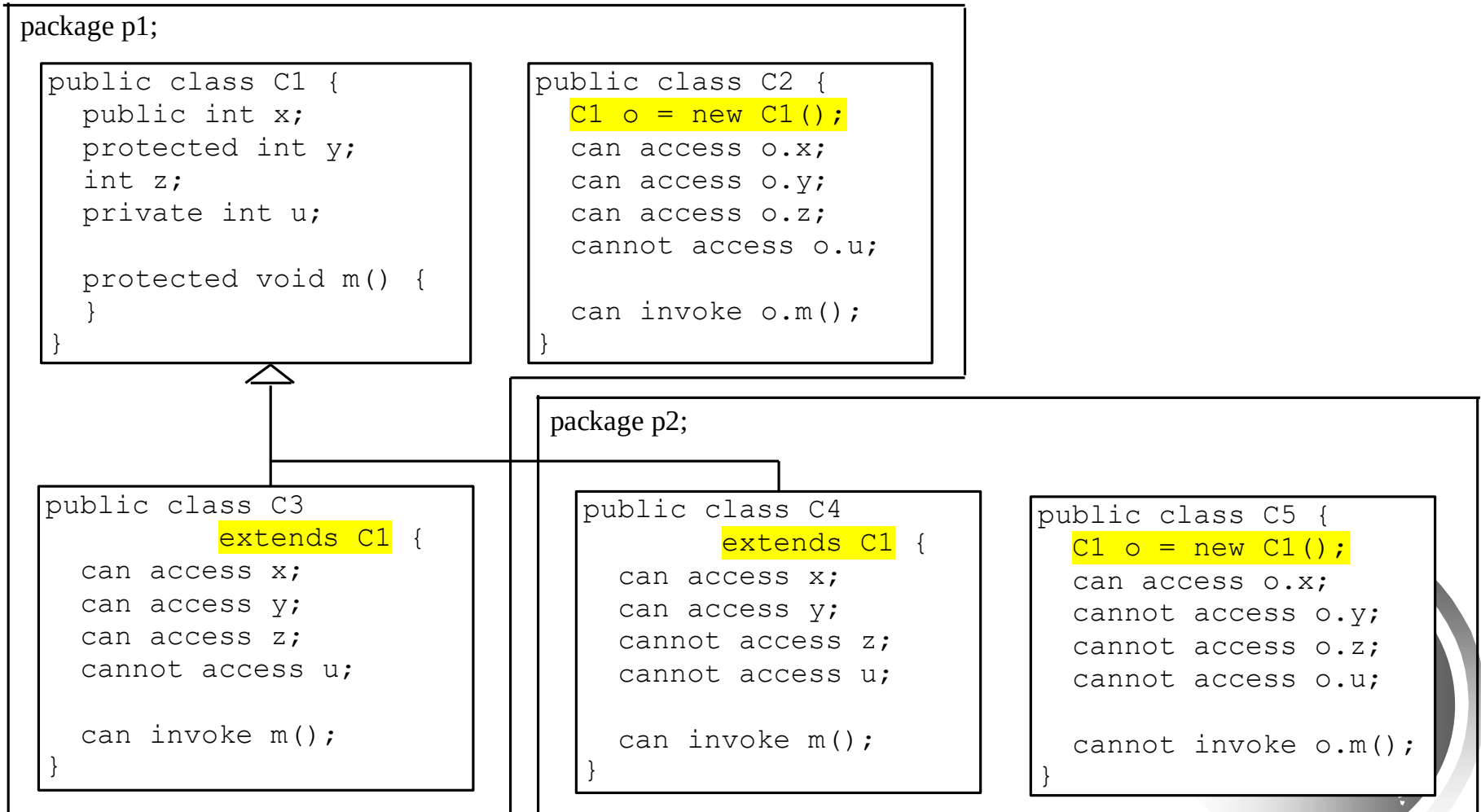


Accessibility Summary

Modifier on members in a class	Accessed from the same class	Accessed from the same package	Accessed from a subclass in a different package	Accessed from a different package
public	✓	✓	✓	✓
protected	✓	✓	✓	–
default	✓	✓	–	–
private	✓	–	–	–



Visibility Modifiers



A Subclass Cannot Weaken the Accessibility

- F A subclass may override a protected method in its superclass and change its visibility to public.
- F A subclass cannot weaken the accessibility of a method defined in the superclass. For example, if a method is defined as public in the superclass, it must be defined as public in the subclass.



NOTES

- You may occasionally want to prevent classes from being extended. In such cases, use the **final** modifier to indicate that a class is final and cannot be a parent class. The **Math** class is a final class.



The `final` Modifier

- ❑ The `final` class cannot be extended:

```
final class Math {  
    ...  
}
```

- ❑ The `final` variable is a constant:

```
final static double PI = 3.14159;
```

- ❑ The `final` method cannot be overridden by its subclasses.



The `final` Modifier

```
final class A
{
    // methods and fields
}
// The following class is illegal.
class B extends A
{
    // COMPILE-ERROR! Can't subclass A
}
```



The `final` Modifier

```
class A
{
    final void m1()
    {
        System.out.println("This is a final method.");
    }
}

class B extends A
{
    void m1()
    {
        // COMPILE-ERROR! Can't override.
        System.out.println("Illegal!");
    }
}
```